Recordings, Not Rows: Lossless Waveform Storage and Lag-Tolerant Retrieval Inside PostgreSQL

Working draft 0.6 — not for distribution

Zin Nam

ABSTRACT

Finding windows in a waveform archive that resemble a query interval up to a small time shift is a bounded-lag cross-correlation search. Exhaustive scans grow linearly with the archive, and standard vector indexes rank by a zero-lag metric that misses shifted matches. Each lag term of a biased, zero-padded cross-correlation is an inner product with a shifted copy of the query, so a small grid of shifted probes through an ordinary inner-product HNSW index, followed by exact re-ranking, recovers the bounded-lag top ten. On seismic stations and a year that no earlier study had read, recall@10 was 0.976 at a p95 of 131 ms, against 2.9 s exhaustively; a zero-lag shortlist of the same size reached 0.61. We implement the method in MorletDB, a PostgreSQL extension that stores recordings as losslessly coded chunks with transactional appends and returns hits as sample intervals that read back bit-exactly. We then measure what bounds its useful range. Post-filtering loses recall under selective metadata filters (0.52 when 1.8% of windows qualify) unless small galleries are routed to exact search and probes are restricted to eligible recordings (0.977–1.0). The vector copy and its index add 1.0–1.4× the raw storage. Maintaining the index at commit sustained 16 channels at 25.6 kHz over a 0.94-million-window archive with a probe p95 of about 0.1 s, once checkpoint settings kept write-ahead logging near three times the data growth. We also report lossless storage of calibrated sensor floats against ALP, Parquet, zstd and FLAC, recovery and access-control tests, and the negative results that shaped the design.

1 INTRODUCTION

Condition monitoring, seismology and clinical physiology produce long, uniformly sampled recordings. An analyst first finds an interval of interest, then asks where similar behaviour occurred and what the original samples show. That workflow needs three things at once: compact storage of the original samples, a search representation, and a reliable link from every hit back to its samples and acquisition context.

Relational systems offer two layouts, each covering part of the problem. One row per sample makes every value addressable but costs about 93 bytes per sample uncompressed in our measurements; compressed time-series extensions shrink this but treat samples as independent floats. A vector per window gives a compact search key, but the application must maintain the link to the source samples, keep representations compatible, and decide what similarity means. Neither layout captures a *recording*: a channel with a sample rate, a unit, a calibration and a half-open sample coordinate system.

MorletDB is a PostgreSQL extension (Rust, via pgrx) built around that unit. Recordings are appended in chunks at global sample offsets; raw chunks, derived representations and checkpoints commit in one transaction; and every search result is a recording identifier plus a sample interval that reads back through the same extension. The design rests on one premise: *authoritative samples and search representations should be stored separately, but connected by explicit interval semantics and transactional ingestion.* The name reflects the first representation tried, Morlet wavelet energies; our experiments later showed no advantage over FFT band powers (Section 8), and the representation is now a versioned, replaceable part.

This paper makes five contributions.

(1) **Shifted-probe retrieval for bounded-lag search** (Sections 6.3–6.4): the identity between lag terms and inner products with zero-padded shifted queries; a diagnosis that zero-lag shortlists fail because of their candidate metric, not because of approximate search; and recovery of the bounded-lag top ten through a standard inner-product index, confirmed on held-out data.
(2) **The costs that bound its operating range** (Sections 6.5–7): filtered search needs routing and filter-aware probes; the index copy costs storage that a single representation can save only at a latency cost; and index maintenance at commit is sustainable at the tested rate only with checkpoint settings that contain write-ahead logging.
(3) **A recording-oriented design inside PostgreSQL** (Section 3): replay-safe chunk appends at global offsets, lossless chunk coding, replaceable search representations and interval read-back, with access control, recovery and upgrades tested end to end (Section 7).
(4) **A storage characterization of calibrated sensor floats** on identical samples against ALP, Parquet, zstd, FLAC and DuckDB (Section 5).
(5) **Negative results that shaped the system**, each from a frozen protocol (Section 8).

We do not claim a new entropy coder, a new similarity measure, or a lag-tolerant index with guarantees: probe search is approximate and evaluated empirically. Code, protocols, raw results and the commit at which each protocol was frozen are public [27].

2 DATA MODEL AND SEMANTICS

A *recording* is a sequence $x_0, \ldots, x_{N-1}$ of float64 values with a sample rate f_s, a channel, a unit, a declared sample quantum q (Section 3.2) and JSON acquisition metadata. An *interval* $[a, b)$ denotes $x_a, \ldots, x_{b-1}$. Sample offsets are authoritative; wall-clock time additionally needs a time origin, and a real acquisition gap starts a new recording rather than silently joining samples.

A search representation $\phi_v(x_{a:b})$ is identified by a versioned extractor name v. Two windows are comparable only if their recordings share sample rate, channel and unit and their windows share length and extractor; metadata containment filters restrict candidates further. These rules reject comparisons that are computable

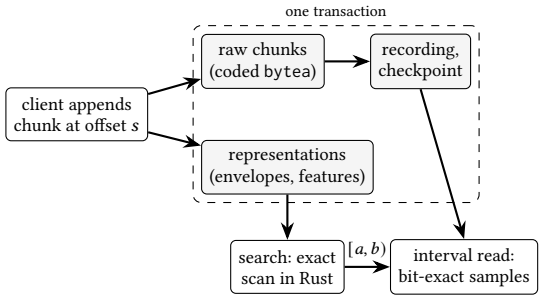


Figure 1: Write and read paths of a collection. Raw chunks, derived representations and the checkpoint commit together; search returns intervals, which are read back from the raw chunks, not from the representation.

but meaningless, such as distances between features from different sample rates.

We separate three notions of correctness that are often conflated. *Sample identity*: decoding returns the appended float64 bit patterns. *Search fidelity*: a stored or approximate representation reproduces a reference ranking, measured here by recall@10. *Task relevance*: retrieved intervals help answer a domain question. Each needs its own evidence; fidelity says nothing about relevance. We call results *source-linked* in the operational sense that each hit carries its recording, interval and metadata; this is not cryptographic provenance.

3 SYSTEM DESIGN

3.1 Collections and transactional appends

A collection is a set of caller-owned tables created by `morlet_create_stream_collection`: recordings with their checkpoint, raw chunks, and per-representation tables (Figure 1). All functions run as `SECURITYINVOKER`, so PostgreSQL privileges and row-level security apply to every read and search. An append names its global start offset:

```
SELECT morlet_start_recording('signals', 'pump7', 25600,
  'accel_x', 'g', '{"asset":"P7"}', 512, 256, 'zscore',
  'none', 0.001);                          -- quantum q
SELECT morlet_append_chunk('signals', 'pump7', 0, :x);
SELECT * FROM morlet_search_envelope_interval('signals',
  'pump7', 81920, 86016, '{"asset":"P7"}', 10);
SELECT samples FROM morlet_read_interval('signals',
  'pump7', 81920, 86016);
```

Raw samples, derived representation rows and the checkpoint commit in one transaction, so a committed checkpoint always refers to committed samples. An append that repeats a committed range with identical samples is verified and accepted as a no-op; a changed retry, a gap or an overlap fails. Collectors can therefore retry after a crash without their own deduplication. Windows of derived representations may cross chunk boundaries.

3.2 Lossless chunk coding

Each raw chunk is one `bytea` value. The codec combines established techniques: fixed polynomial predictors of order 0–4 as in Shorten and FLAC [5, 6], partitioned Rice coding of zig-zag mapped residuals [7], and a CRC-32C checksum. Its distinctive part is how it treats floats.

Most sensor values are calibrated integers: a digitizer emits counts d, and acquisition software stores $x = f(d)$, for example $x = d \cdot s$ (seismic counts, WAV) or, in EDF [8], $x = (d - d_{\min})(p_{\max} - p_{\min})/(d_{\max} - d_{\min}) + p_{\min}$. The recording declares a nominal quantum q: 1 for counts, 0.001 for three decimals, the ADC step for calibrated values. Per chunk, the encoder seeks a grid

$$x_i = o + k_i\, s \;\oplus\; \delta_i, \tag{1}$$

with integer indices k_i, a step s refined from q to within one unit in the last place (ulp), an offset o (format version 3; zero in version 2), and corrections δ_i, stored as differences of order-preserving integer images of float64, so that decoding reproduces every bit pattern. A division mode $x = k/d$ covers decimal steps exactly. Indices are predicted and Rice-coded like integers; corrections form a second stream. If no grid fits within a thousandth of a step, the chunk is coded as order-preserving float bits, and if that is not smaller, as raw float64. The quantum is a storage hint, never a conversion: a wrong quantum costs space, not accuracy.

3.3 Materialized, replaceable search representations

Search representations are rows derived from raw chunks and keyed by recording, window size and start sample. We study the envelope representation: for a 4096-sample window, z-score, take the magnitude of the analytic signal, smooth with a 50-sample moving mean, keep every tenth value, and z-score again. This gives 410 values, stored as float16 (821 bytes with a version byte). The distance between a query envelope q (full precision) and a stored envelope g is

$$D(q,g) = 1 - \max_{|\ell| \le L} \frac{1}{n} \sum_t q_{t+\ell}\, g_t, \qquad L = \lfloor 0.075\, n \rfloor = 30, \tag{2}$$

a biased cross-correlation over bounded shifts. Search is an exhaustive scan of eligible envelopes scored in Rust (`morlet_envelope_distance`, eight lags per inner block), issued as one statement that PostgreSQL can run with parallel workers, with the source interval excluded and overlapping windows of one recording suppressed greedily in rank order. Materialization and float16 storage change values relative to envelopes computed from raw samples; Section 6 measures the effect. Dropping and rebuilding a representation never touches raw chunks.

Probe search. Each lag term of D is an inner product: with the zero-padded shifted query $s_\ell[t] = q_{t+\ell}$ for $0 \le t + \ell < n$ (else 0), $\sum_t q_{t+\ell} g_t = \langle s_\ell, g \rangle$, so $1 - D(q,g) = \max_\ell \langle s_\ell, g \rangle / n$. An optional, pgvector-backed table holds a float16 copy of the envelopes with an inner-product HNSW index. `morlet_search_envelope_probes` issues probes at lags $-L, -L{+}5, \ldots, L$ (13 at $n = 410$), takes 50 candidates per probe, and re-ranks their union with the exact scorer under the same eligibility, exclusion, ordering and suppression; returned distances are exact, only the candidate set is approximate. Probes must use inner product rather than cosine, because $\|s_\ell\|$ differs between lags. `morlet_build_envelopes` writes each new window's vector row in the same transaction, and rows reference their envelopes with cascading deletes. pgvector remains optional:

Table 1: Studies and data reuse. Data slices are specified in the text.

Study	Workload	Data
P2	DB storage vs TimescaleDB; 4 inputs × 4.2 M samples	reused
B1	codec vs ALP, Parquet, zstd, FLAC, DuckDB; 6 inputs	1 new input
R1b	envelope precision; 1,826 windows, 1,557 queries	unread slice
P1	the same envelopes in pgvector	R1b's
R2	search scaling to 235 k windows; zero-lag shortlist	99.9% unread
R2-D	diagnostic: exact vs HNSW shortlists	R2's (consumed)
BL1	shifted probes, development	R2's (consumed)
BL1-C	shifted probes, confirmation; 234,622 windows	unread stations and year
BL1-S	maintenance during replayed ingestion	R2's (consumed)
SS1	one stored envelope representation	R2's and BL1-C's
SS3	filtered search, routing; 5 filter levels	R2's; BL1-C's for confirmation
CW1	external workers vs in-database building	R2's (consumed)
S1	sleep staging by retrieval; 39 held-out subjects	held-out split

the objects that use it are created on demand. Filters are handled by routing: when at most a threshold of windows (default 5,000) is eligible, the exact scan runs; otherwise each probe scans only eligible recordings with pgvector's iterative index scan (Section 6.5).

4 EVALUATION METHOD

Every study follows the same discipline: a protocol with questions, data, arms, metrics and decision thresholds is committed before data is read or measured; harness and tests are committed before the run; each study runs once; deviations are logged in the protocol. Table 1 lists the studies used here and which data each reused.

All runs used one Apple M5 Max (macOS, 128 GB), PostgreSQL 17.11 built by pgrx, pgvector 0.8.0 and one client over localhost. TimescaleDB 2.30.2 ran in Docker; we compare its sizes, not its speed.

5 STORAGE

5.1 Database footprint (P2)

P2 appended the same 4,194,304 samples of four inputs (CX seismic counts [23], IMS bearing vibration [26], Paderborn bearing vibration [25] as ADC counts and as physical floats) to MorletDB and to TimescaleDB hypertables with native compression (Gorilla for floats; delta-of-delta and Simple-8b for integers), and measured total on-disk bytes, including indexes, TOAST and metadata tables, after verifying that four intervals per layout read back bit-exactly [28].

TimescaleDB on integer columns comes closest but is 24–35% larger (Table 2), and requires the user to convert values to counts

Table 2: P2: total database storage in bits per sample. T: the TimescaleDB layouts tested (a row per sample with an integer or float column; a row per 4096-sample array).

Input	MorletDB	T int	T float	T array
CX seismic counts	**6.12**	8.25	14.34	25.0
IMS three-decimal	**11.36**	14.06	71.95	28.1
Paderborn counts	**9.95**	12.89	24.58	26.4
Paderborn physical	**11.53**	—	70.12	29.5

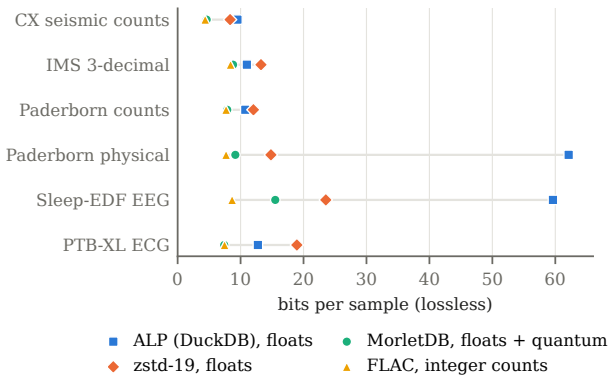


Figure 2: B1: lossless bits per sample on identical samples. Float codecs see only float64 values; MorletDB also sees the nominal quantum; FLAC sees integer counts. Table 3 gives all arms.

first. On float columns, Gorilla's XOR coding *expands* noisy decimal and calibrated values beyond 64 bits per sample. We tested one `segmentby`/`orderby` configuration and not TimescaleDB's newer column formats. Of MorletDB's footprint, the codec payload is 4.6–9.2 bits per sample (Table 3); PostgreSQL row and TOAST overhead adds 1.5–2.6 bits per sample at 4096-sample chunks.

5.2 Codec against modern baselines (B1)

B1 compares codecs on identical samples, each verified lossless [29]. Float codecs receive only float64 values: zstd level 19 per chunk, Parquet with `BYTE_STREAM_SPLIT` and zstd, and DuckDB 2.0's automatic float compression, which uses ALP and its fallback ALP-RD [1, 13]. MorletDB receives the values plus the nominal quantum. Integer codecs receive the exact digitizer counts: FLAC [6], Parquet delta coding with zstd, and DuckDB bit-packing. Two inputs extend the earlier storage inputs: a Sleep-EDF EEG slice [20, 21] from 32 nights with symmetric and asymmetric calibration ranges (an earlier engineering check had measured MorletDB's own storage of this collection, not the other arms), and PTB-XL ECG lead I [22], new to the storage studies. The Parquet float arm is one fixed encoding, not a tuned Parquet baseline; ALP is DuckDB 2.0.0's implementation, measured as table blocks.

Figure 2 and Table 3 show three regimes.

Counts and decimals. When values are integers or short decimals (CX, IMS, Paderborn counts, ECG with gain 1000), ALP and

Table 3: B1: bits per sample. Z: zstd-19; PQ: Parquet; DDB: DuckDB.

	float input				integer input		
Input	Z	PQ	ALP	**MorletDB**	FLAC	PQ	DDB
CX counts	8.33	6.84	9.50	**4.61**	4.38	5.24	6.38
IMS 3-decimal	13.24	42.94	11.00	**8.81**	8.41	9.87	10.88
Paderborn counts	12.04	9.48	10.75	**7.88**	7.70	9.61	11.25
Paderborn physical	14.82	30.57	62.13	**9.17**	7.70	9.61	11.25
Sleep-EDF EEG	23.54	38.15	59.63	**15.51**	8.65	10.22	12.00
PTB-XL ECG	18.94	44.64	12.75	**7.40**	7.46	10.27	12.75

MorletDB both recover integers; MorletDB is 20–51% smaller than ALP, plausibly because consecutive samples are correlated and prediction exploits that, while ALP's frame-of-reference bit-packing does not; B1 does not isolate this. MorletDB is within 5% of FLAC, whose LPC stage it lacks, and slightly below FLAC on ECG.

Calibrated floats. On Paderborn physical values and Sleep-EDF, no decimal exponent turns values into integers, and the tested DuckDB ALP implementation selected ALP-RD, storing about 60 bits per sample, near raw. Of the three metadata-free arms, zstd was smallest. MorletDB, which additionally receives the nominal quantum, was 1.5–1.6× smaller than zstd. Our pre-registered claim of at least 2× was not met.

The correction gap. Given counts, MorletDB codes the two calibrated inputs at 7.9 and 8.8 bits per sample; given the calibrated floats, it needs 1.3 and 6.7 bits per sample more. That extra cost, relative to MorletDB's own integer-input path, is the correction stream: the source software computed x with a particular sequence of roundings (on Sleep-EDF a multiplication, a division and an addition), which a single fitted step and offset does not reproduce for every sample. It does not explain the whole difference from FLAC, which also includes MorletDB's integer path being 0.15–0.18 bits per sample larger than FLAC's. Replaying the source's calibration arithmetic might remove the corrections when the correct parameters and operation sequence are known; this is neither implemented nor tested. Our pre-registered claim that MorletDB stays within 15% of the best integer codec held on four of six inputs and failed on these two.

For float compression, these inputs illustrate that real-valued sensor data can be neither decimal nor arbitrary but an affine image of integers under floating-point rounding. On the two such inputs we tested, the decimal-oriented ALP implementation fell back to near-raw size and the best byte-oriented arm was 1.5–1.6× larger than coding with a declared quantum.

6 RETRIEVAL

6.1 Precision of materialized envelopes (R1b)

R1b selected an envelope precision on 2023 seismic windows and evaluated it once on 2022 event and noise windows that, according to its report, no earlier study had read: 1,826 windows at ten CX stations [23], 1,557 event windows from 284 catalogued events and 269 no-match distractors [30]. The reference decodes raw samples and computes envelopes in the client.

Table 4: R1b on 2022 windows. Size relative to the raw chunk table; latency includes the client path.

Envelopes	Size/raw	Recall@10	Same order	p95 ms
from raw (reference)	—	1	1	1324
float32	28.1%	1.000	1.000	14
float16	**12.9%**	**0.9999**	**0.987**	**16**
uint8	7.2%	0.998	0.845	17
uint8, coarser	4.2%	0.952	0.067	16

Float16 envelopes met all frozen criteria (Table 4): recall@10 of 0.9999, 12.9% of the raw table, and an unchanged catalogue-distance relevance proxy (+0.001 km, 95% interval $[-0.002, +0.004]$). The roughly 80-fold latency reduction measures materialization against a client that decodes and recomputes; it is not a speedup over an optimized raw-scanning engine.

6.2 Same storage, different query (P1)

P1 stored the identical float16 envelopes in a pgvector `halfvec(410)` column [11]. Both tables occupied exactly 1,794,048 bytes, and all 1,826 envelopes were bit-identical, so envelope storage itself is no advantage. Zero-lag cosine search, however, shared only 64.7% of the lag-tolerant top ten and reproduced its order for 0.1% of queries. Scoring lags in the client on the same `halfvec` rows reproduced MorletDB's ranking exactly. In a later same-run comparison, median latency was 6.85 ms for MorletDB, 0.49 ms for zero-lag cosine and 20.3 ms for client lag scoring. The first two answer different questions, so their ratio is not a like-for-like speed comparison.

6.3 Scaling and zero-lag shortlists (R2, R2-D)

R2 asked how exact search scales and whether a standard approximate index can serve as its first stage [31]. We loaded CX HHZ at stations PB01, PB02, PB06 and PSGCX for 1–14 March 2025, downloaded as continuous station-days for R2 (48 short windows, 0.1% of these samples, had been read by an earlier study): 481 M samples in 128 gap-free recordings, appended in 451 s and stored at 6.2 bits per sample. Envelopes over 4096-sample windows at stride 2048 gave 234,762 windows (275 s to build). Galleries were nested by day, from 4,217 to 234,762 windows. The 100 queries were 50 random and 50 high-energy windows from the first day. A shortlist arm takes the top K windows by zero-lag cosine on a pgvector `halfvec` copy of the same float16 envelopes and re-ranks them with the exact lag-tolerant distance in the database, under the same exclusion and suppression. R2 intended its shortlist to come from an HNSW index (m 16, `ef_construction` 64), with a pre-registered bar of recall@10 $\geq$ 0.95 at no more than a tenth of the exact p95 latency on the largest gallery. A diagnostic follow-up on the same data (R2-D, protocol frozen before measuring) found that R2's statement never used the index: its query vector came from a subquery, which pgvector cannot use as an index ordering key, so PostgreSQL computed an exact zero-lag top K. R2-D therefore compares an explicitly exact shortlist (E-K, plan-checked sequential scan) with the HNSW index actually used (H*-K, query vector as a parameter, plan-checked index scan).

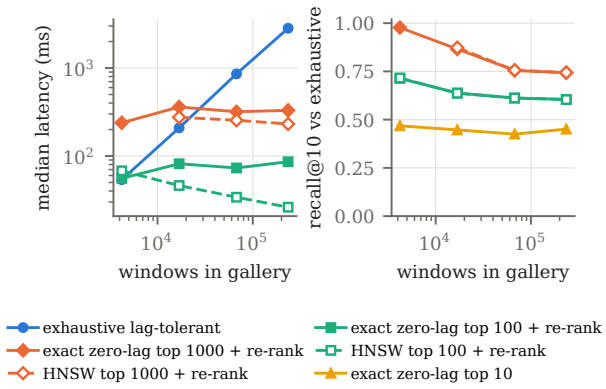


Figure 3: R2 and R2-D on CX data, 1–14 March 2025. Left: median latency of the exhaustive lag-tolerant scan and of zero-lag shortlists (exact: solid; HNSW: dashed) followed by exact re-ranking. Right: recall@10 against the exhaustive top ten; HNSW lies on the exact shortlist. H*-1000 is missing at the smallest gallery, where the planner did not use the index. 100 queries per gallery; one client, warm cache.

Over the measured range, exact search time grew linearly at 12.1–12.8 µs per stored window (Figure 3): 54 ms for 4,217 windows, 0.86 s for 67,472 and 2.8 s for 234,762. These figures are for the original implementation, which iterated over candidates with a PL/pgSQL cursor that PostgreSQL never parallelizes; fetching the best rows in one parallel statement and suppressing overlaps on those (fetching more if suppression exhausts them) later returned identical lists with bitwise-equal distances about 2.5× faster (unfiltered p50 3.3 to 1.3 s on a loaded machine; one station 0.78 to 0.31 s). Interpolating between the two smallest galleries places the 100 ms mark near 8,000 windows; no gallery of that size was measured.

No shortlist arm met the bar (Figure 3). With K=1000, recall@10 fell from 0.977 on the smallest gallery, where 1000 candidates are a quarter of it, to 0.743 on the largest; with K=100, from 0.714 to 0.605. The HNSW shortlist agreed with the exact one on 98–100% of candidates on the largest gallery and gave the same recall to within 0.007 at every size measured (0.744 for H*-1000). Every shortlist returned K candidates, and every member of the exhaustive top ten that a shortlist contained survived re-ranking, exclusion and suppression, so coverage equalled recall. The loss is therefore attributable to the zero-lag candidate metric at these budgets, not to approximate search or to the suppression rules: a quarter of the lag-tolerant top ten lies outside the 1000 nearest windows by zero-lag cosine. The exact zero-lag top ten alone shared 43–47% of the exhaustive top ten on these continuous recordings, below P1's 64.7% on event windows. These are measurements for two budgets and one HNSW configuration.

With the index in use, the HNSW top 100 plus re-ranking took a median 26 ms on the largest gallery, against 2.8 s exhaustively, at recall 0.60; the top 1000 took 0.23 s at recall 0.74. Re-ranking dominates the latter.

Representation overhead. The materialized envelopes took 12.9% of the raw chunk table in R1b but 60.2% in R2. Measured sizes account for the factor of 4.68: halving the stride from 4096 (one envelope per event window) to 2048 doubles the number of envelopes per sample (×2.0); R2's continuous, mostly ambient recordings compress to 5.04 payload bits per sample against 12.39 for R1b's event windows (×2.46); row and TOAST overheads differ slightly (×0.95 combined). The ratio thus depends on stride, raw compressibility and physical layout, and the 12.9% figure does not transfer to continuous archives. R2's benchmark held four structures side by side: raw chunks (371 MB, always required), envelopes (223 MB, used by the exhaustive scan and every re-rank), the `halfvec` copy (241 MB) and its HNSW index (275 MB). The copy stores the same values as the envelopes, so a deployment would keep one of the two if its scorer read that format (MorletDB's reads its own); the index is needed only for an index-based shortlist.

Table 5: BL1-C, held-out confirmation: 234,622 windows, 100 queries (50 random, 50 high-energy).

	Exhaustive	Parallel exact	**13 probes × 50**
Recall@10	1	1	**0.976**
p50 / p95 (ms)	2,834 / 2,854	698 / 713	**97 / 131**
Windows re-ranked	234,622	234,622	357
Added storage / raw	—	—	1.00

6.4 Shifted-probe search (BL1)

BL1 asked whether candidate generation that respects the lag range recovers the exhaustive top ten [38]. Its protocol fixed grids of 7 and 13 probes (spacing 10 and 5) and 50 or 100 candidates per probe, re-ranked by the unchanged scorer, with controls that separate descriptor quality from approximate search: the same probes scored exactly (no ANN), and an exact zero-lag shortlist given the same number of candidates. The gate required recall@10 ≥ 0.95, p95 at most a third of the exhaustive scan's and below the strongest exact baseline, and added storage at most 1.5× the raw chunk table. The strongest exact baseline was a single parallel SQL statement scoring every eligible envelope, 3–4× faster than the PL/pgSQL scan with identical results.

On R2's development data all four configurations passed; the selected one (13 probes, 50 each) reached recall 0.998 at a p95 of 115 ms against 2,835 ms exhaustively and 954 ms for the parallel exact scan, re-ranking 338 windows on average. We froze it and evaluated it once on CX stations PB05, PB11, PB14 and PB18, 1–14 March 2021: stations and a year that no earlier study in the repository had read, chosen from the availability service alone before any waveform was downloaded.

All confirmation criteria held (Table 5). The result was weaker than on development data in one respect: probes scored exactly reached 0.999, so HNSW cost 0.023 recall here (0.002 before), mostly on high-energy queries (0.954 against 0.998 for random ones). The equal-budget zero-lag control reached 0.613, and Figure 4 shows why: half the exhaustive top-ten items score best beyond ±5 lags, and zero-lag candidates miss them while the probes do not. The MorletDB functions themselves, run on the same queries, reached recall 0.977 at a p95 of 50 ms, faster than

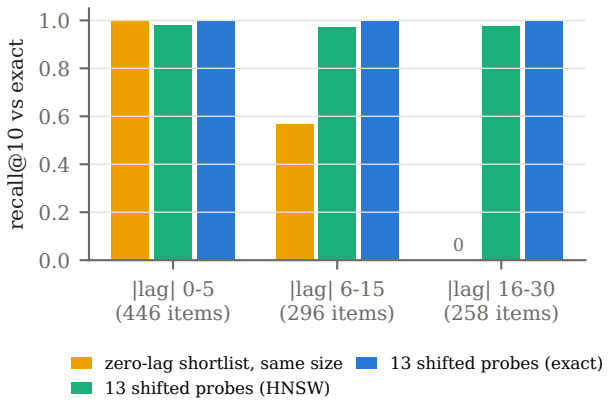


Figure 4: BL1-C: recall of the exhaustive top ten by the lag at which each item scores best. A zero-lag shortlist of the same size as the probe union finds aligned matches but none beyond ±15.

the experiment's client-driven probes because the probe loop runs inside the server. A first product version used a smaller HNSW search breadth than the evaluated configuration and reached 0.963; it was aligned to the evaluated setting.

During ingestion (BL1-S). Replaying one day of four stations (16,868 windows) with maintenance in the append transaction made every window searchable at its commit while appends ran at 919,000 samples per second, 53% of raw-only throughput. At 200× real time, probe queries during ingestion took a p95 of 82 ms against 189 ms for the exact scan, with recall 0.9997 against the exact scan on the same snapshot. A background worker instead kept up at that pace (p95 delay 0.09 s) but fell behind unpaced appends at about 200 windows per second and drained a 13,284-window backlog in 64 s; after it was killed and restarted, every window had exactly one envelope and one vector row. These runs used development data and one machine.

6.5 Filtered search (SS3)

Investigations usually restrict the gallery: one sensor, one period. The first probe implementation applied eligibility after probing, so a selective filter discarded most of its 650 candidates. SS3 measured three strategies at five filter levels per query (one station-day, 1.8% of windows; one station over four days, 7.2%; one station, 25%; four days of all stations, 29%; unfiltered) [40]: exact search, post-filtered probes, and filter-aware probes that restrict each index scan to eligible recordings with pgvector's iterative scan. A router, fixed on development data before the confirmation run, sends a query to exact search when at most 5,518 windows are eligible (where exact p95 reached 100 ms) and to filter-aware probes otherwise.

Post-filtering lost up to half the top ten (Table 6). Filter-aware probes restored recall but were slow at the smallest level (p95 724 ms), where the planner abandoned the index; exact search, at 68 ms there, is the right path, and the router chooses it. No returned row violated its filter, and with row-level security restricting a role

Table 6: SS3 on the BL1-C data: mean recall@10 against exact search, 100 queries per level. Routed: the product function.

Eligible windows	Post-filter	Filter-aware	Routed	Routed p95
1.8% (4,217)	0.515	0.998	**1.0** (exact)	85 ms
7.2% (16,868)	0.848	0.997	**0.997**	207 ms
25% (58.6 k)	0.972	0.986	**0.986**	98 ms
29% (67.5 k)	0.976	0.994	**0.994**	88 ms
100% (234.6 k)	0.976	0.977	**0.977**	69 ms

to one station every strategy returned only that station and rejected a hidden recording as the query source. The router met its recall and safety conditions at every level on both databases; it first missed its latency bound because counting eligible windows cost about 20 ms, and after counting from the recordings table instead it missed only at the unfiltered level of the confirmation data, by 0.8 ms. The product function counts exactly but stops at the threshold, and reproduced the router's recall at every level.

6.6 Storage of search representations (SS1)

With probe search enabled every window is stored twice: as the scorer's `bytea` envelope and as the index's `halfvec` copy, which hold the same float16 values in 98.3% of positions. SS1 tested whether one copy suffices [39]. A single `halfvec` table holding the stored bits was exact on both databases (no mismatching value among 469,384 windows, no unequal distance among about 9.4 million pairs, identical top-ten lists) and saved about 250 MB per 235,000 windows, 0.49–0.67× the raw data, but made exact scans 23% slower through per-row format conversion. An HNSW index on an expression over the existing envelope table kept exact search unchanged and was equally exact, but raised probe p95 by 27–32% (about 20 ms), most likely because the expression is re-evaluated for every returned row. Both missed their pre-registered latency bounds, so the two-copy layout remains the default; the expression index is a valid storage-saving operating point when 20 ms matter less than 250 MB.

6.7 Building representations (CW1)

As a first step toward compaction-style workers that run representation work outside the database, CW1 compared building envelopes inside PostgreSQL with stateless external processes that read encoded chunks through SQL, compute with the same Rust code, and write rows back with key-idempotent inserts [42]. On 33,736 windows the external path produced byte-identical rows in every run, survived a killed and restarted worker and two workers processing the same recordings, and moved 90% of the per-window CPU off the database server (0.13 against 1.31 ms of database CPU per window). It also used about four times less CPU in total, which exposed the real inefficiency: the in-database builder issued one SQL statement per window and decoded every raw sample twice. Batching the in-database builder, so that each chunk is decoded once for up to 256 windows, reduced database CPU to 0.31 ms per window and raised one session from 740 to 3,084 windows per second (four sessions: 12,034), byte-identical to before. External workers remain a viable basis for separating compute from storage, but

on one machine batching delivered most of their benefit without them.

7 OPERATING THE SYSTEM

Before a bounded pilot with human review of results, we tested access control, recovery and sustained load against plans frozen before each run [41]. All runs shared the machine with an unrelated CPU-intensive job (load average 3–20), so latencies are indicative; correctness results are not affected.

Access control. With row-level security only on the recordings table, a tenant could still read other tenants' chunks, envelopes and probe vectors by querying the companion tables directly: search functions honoured the policy, the tables did not. `morlet\_enable\_row\_security` now applies a policy to every companion table that admits rows whose recording is visible. Its first form, a correlated `EXISTS`, prevented parallel plans and tripled exact search time (946 to 2,821 ms on 235,000 windows); comparing against an array of visible recording identifiers kept the parallel scan (995 ms, +5%) and cost probe search 6 ms. We did not test policies with thousands of visible recordings.

Recovery. On a 50,604-window database every check passed: each recording hashed identically afterwards, each complete window had exactly one envelope and probe row, and exact searches returned identical lists. A full dump restored in 18 s; dumping only recordings and chunks took a third of the space (146 against 459 MB), and rebuilding envelopes and the probe index after restore took 23 s. Twenty backends terminated at random points inside append transactions left no partial rows, and replaying the chunk succeeded; aborted builds resumed, and an aborted probe enable could be retried. Killed and duplicated external workers produced exactly one byte-identical row per window, but did not write probe rows, so their windows stayed invisible to probe search until filled. After deleting 15% of recordings, `VACUUM` took 113 s and left the index size unchanged, whereas an online `REINDEXCONCURRENTLY` took 10 s while four clients searched (p95 23 to 31 ms). An extension update forced to fail midway rolled back completely.

Sustained load. On an archive of 938,768 windows (4.8 GB), sixteen channels at 25.6 kHz (409,600 samples/s) were appended by four writers, each building envelopes and probe rows in the append transaction, while four analysts issued one query every two seconds, half filtered to about 2% of windows and half unfiltered. With PostgreSQL's default configuration the run met its criteria (filtered p95 167 ms; probe p95 337 ms, spot-check recall 0.998; ingest lag at most 22 s, recovered), but wrote 13.8 GB of write-ahead log for 0.76 GB of database growth in 15 minutes: HNSW insertions touch pages across a 1.2 GB index, and each checkpoint forces full-page images of them. At twice the rate the writers fell behind without recovering; capacity was about 1.3–1.4× the target. With a pilot configuration (8 GB shared buffers, 16 GB `max\_wal\_size`, 15-minute checkpoints, WAL compression, changed together) the same run wrote 1.91 GB of WAL for 0.65 GB of growth, kept ingest lag at or below 2 s, and gave filtered and probe p95 of 81 and 100 ms (recall 0.994); background load was lower in this run (7.3 against 11.7), which confounds the latency comparison but not the WAL volume. Unfiltered exact search over the full archive stayed at 7.8–9.7 s p95, which is why the router sends large galleries to probes. A two-hour stability run at the target rate is pending.

8 APPLICATION EVIDENCE AND NEGATIVE RESULTS

Sleep staging by example (S1). On 153 Sleep-EDF nights with a subject-disjoint split, staging each 30-second epoch by the 40 most similar expert-scored epochs from other subjects gave a mean per-subject Cohen's κ of 0.583, against 0.541 for logistic regression on the same features (+0.042, 95% CI [0.016, 0.068]), with equal macro-F1 [32]. Retrieval thus exposes source-linked neighbours at no measured accuracy cost against this simple comparator; part of the κ difference comes from the regression's class-balanced weights. The result is far from state-of-the-art staging and makes no clinical claim.

Morlet versus FFT. In the same study, FFT band powers with the same band layout gave κ 0.589 against 0.583 for Morlet energies. We found no advantage for the wavelet representation and made the extractor replaceable.

Wavelet and progressive storage. A compression benchmark found the tested wavelet decomposition codec 19–57% *larger* than scalar quantization with conventional coding at matched error bounds, and progressive raw storage missed its byte-saving gate (0.59 against ≤ 0.5 of a cascade) because a separate envelope index dominated [33]. This motivated storing a retrieval representation *next to* losslessly coded raw data rather than a multiresolution raw format.

Reference coding. Coding a machining recording as a residual against an earlier execution of the same operation would have saved at most 0.0019% on the Bosch CNC data [24], because executions were too dissimilar sample by sample [43].

Out of scope. Three later studies tested retrieval-based forecasting of bearing and turbofan degradation and current-to-vibration estimation; none passed its gate [27]. They concern analytics on top of the store rather than storage and retrieval.

9 RELATED WORK

Float compression. Gorilla [3] and Chimp [4] XOR consecutive values and suit slowly changing metrics; ALP [1] finds decimal exponents and falls back to ALP-RD for high-precision doubles; FCBench [2] compares such methods across domains. Our results add a data class to this picture: calibrated sensor floats. On the two such inputs we tested, a declared quantum with per-chunk step refinement was smaller than the tested ALP implementation and than zstd; whether replaying calibration arithmetic can remove the remaining corrections is open.

Waveform and audio coding. Shorten and FLAC [5, 6] established polynomial or LPC prediction with Rice coding [7]; seismology stores counts with Steim compression in miniSEED [9]. MorletDB reuses these techniques and adds the float grid with exact corrections.

Time-series and array stores. TimescaleDB [10] compresses rows of samples in PostgreSQL, with Gorilla-style coding for floats. SciDB [15] and TileDB [16] manage dense arrays outside a relational engine. Columnar formats such as Parquet [14] and engines

such as DuckDB [13] store sample columns well (Table 3) but leave recording semantics, replay-safe appends and interval read-back to the application.

Similarity search. FAST [17] fingerprints seismic waveforms for large-scale similarity search; the UCR suite [18] and matrix-profile methods [19] search subsequences under shift and warping. Graph indexes such as HNSW [12], available in PostgreSQL through pgvector [11], give approximate nearest neighbours under fixed metrics. k-Shape's shape-based distance [34] and MASS [35] evaluate normalized cross-correlation over all shifts exactly, the latter with per-subsequence normalization unlike MorletDB's biased, bounded-lag score; FAST obtains shift tolerance on the data side by indexing densely overlapping windows. Streaming vector indexes such as FreshDiskANN [36] and VStream [37] keep zero-lag ANN indexes fresh under updates. Among the sources we read we found none that issues query-side shifted probes into an inner-product ANN index to recover a bounded-lag cross-correlation top-k; we do not claim the general idea of multi-probe search as new. MorletDB's contribution is the placement of an exact lag-tolerant scorer inside the database next to the samples it links back to, a candidate-generation method for it that uses standard indexes, and their measurement against these alternatives.

10 LIMITATIONS

One machine, warm caches. All timings come from one laptop-class machine with warm caches, most with one client; the load tests of Section 7 used four writers and four analysts on a machine shared with an unrelated workload, and we did not measure disk-cold caches, several tenants under load, or archives beyond about one million windows. R2 and R2-D tested one HNSW configuration and two candidate budgets; R2's own HNSW arm silently ran as an exact scan because no plan check covered it, which R2-D found and corrected. Probe search was evaluated on galleries of about 235,000 windows; filtered search was evaluated at five filter levels on the same two databases, and the router missed its pre-registered latency bound at the unfiltered level of the confirmation data by 0.8 ms. Stage B used one day of four channels and one worker. Storage sizes are deterministic and transfer to other machines; latencies do not.

Baselines. TimescaleDB was tested in one configuration; ALP was measured inside DuckDB, including its block layout; FLAC is the only LPC-based integer codec. The Parquet float arm used one fixed encoding.

Data reuse and relevance. P1 reuses R1b's data, the P2 inputs come from earlier studies, an earlier engineering check had measured MorletDB's storage of the Sleep-EDF collection, and R2-D is a diagnostic on R2's data. R1b's 2022 windows, BL1-C's March 2021 slice of four never-used CX stations and S1's held-out subjects had not been used before; R2's March 2025 slice overlapped 0.1% of its samples with windows an earlier study had read. Our main retrieval metric is fidelity (recall against an exact reference). Task relevance is supported only by a catalogue-distance proxy for seismic events and by sleep staging; we make no claim about other domains or workloads.

Correctness. Bitwise round trips on every measured chunk, replay tests and permission checks are engineering evidence, not formal verification of the codec, the transaction logic or access control.

11 CONCLUSION

Treating a recording, rather than a sample or a vector, as the unit of storage lets a relational engine keep original samples compact, keep search representations replaceable, and return results that read back exactly. On the two calibrated inputs tested, declaring the digitizer grid mattered more than codec sophistication: where the tested ALP implementation stayed near 60 bits per sample, MorletDB needed 9–16, and most of the remaining gap to integer codecs is the cost of reproducing the source's rounding. Whether replaying calibration arithmetic removes it is untested. For retrieval, an ordinary vector column stores envelopes equally well, but lag tolerance changes the answer: over the measured range the exhaustive scan grew linearly, and a zero-lag shortlist of 1000, exact or from HNSW, recovered three quarters of its top ten. Probing the lag terms directly, as inner products with shifted queries, recovered 97.6% of the bounded-lag top ten on held-out data at a twentieth of the exhaustive latency, through an index PostgreSQL users already have, and kept new windows searchable at commit. Materialized representations are cheap relative to raw data only when windows are sparse or raw data compresses poorly. Two engineering results matter as much as the algorithm: filters need routing, because post-filtering an approximate candidate set silently loses recall when few windows qualify; and building representations in batches, decoding each raw chunk once, cut database CPU per window 4.2× with byte-identical output, which is most of what moving the work to external compute workers would have bought. Keeping a single stored representation is exact and saves a third of the search storage, but on PostgreSQL either design costs 20–30% latency on one search path. Operating the index is part of the method's cost: maintaining it at commit sustained the tested ingest rate only once checkpoint settings contained write-ahead logging, and access control had to cover every companion table, not only recordings. Exact replay of calibration arithmetic, relevance beyond scorer fidelity, and long-run stability remain open.

ARTIFACT AVAILABILITY AND DISCLOSURE

Code (Apache-2.0), frozen protocols with their commits, harnesses, tests and raw result files are at https://github.com/merkhet-io/morletdb; datasets keep their own licences. This draft was prepared with AI assistance from the repository's reports; the author must audit text, numbers and references before submission.

REFERENCES

[1] Azim Afroozeh, Leonardo X. Kuffo, and Peter Boncz. 2023. ALP: Adaptive Lossless floating-Point Compression. *Proc. ACM Manag. Data* 1, 4, Article 230 (Dec. 2023), 26 pages. doi:10.1145/3626717.

[2] Xinyu Chen, Jiannan Tian, Ian Beaver, Cynthia Freeman, Yan Yan, Jianguo Wang, and Dingwen Tao. 2024. FCBench: Cross-Domain Benchmarking of Lossless Compression for Floating-Point Data. *Proc. VLDB Endow.* 17, 6 (2024), 1418–1431. doi:10.14778/3648160.3648180.

[3] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang, Justin Meza, and Kaushik Veeraraghavan. 2015. Gorilla: A Fast, Scalable, In-Memory Time Series Database. *Proc. VLDB Endow.* 8, 12 (2015), 1816–1827. doi:10.14778/2824032.2824078.

[4] Panagiotis Liakos, Katia Papakonstantinopoulou, and Yannis Kotidis. 2022. Chimp: Efficient Lossless Floating Point Compression for Time Series Databases. *Proc. VLDB Endow.* 15, 11 (2022), 3058–3070. doi:10.14778/3551793.3551852.

[5] Tony Robinson. 1994. SHORTEN: Simple lossless and near-lossless waveform compression. Technical Report CUED/F-INFENG/TR.156, Cambridge University Engineering Department.

[6] Martijn van Beurden and Andrew Weaver. 2024. Free Lossless Audio Codec (FLAC). RFC 9639, RFC Editor. doi:10.17487/RFC9639.

[7] Robert F. Rice. 1979. Some Practical Universal Noiseless Coding Techniques. JPL Publication 79-22 (NASA-CR-158515), Jet Propulsion Laboratory.

[8] Bob Kemp, Alpo Värri, Agostinho C. Rosa, Kim D. Nielsen, and John Gade. 1992. A simple format for exchange of digitized polygraphic recordings. *Electroencephalography and Clinical Neurophysiology* 82, 5, 391–393. doi:10.1016/0013-4694(92)90009-7.

[9] International Federation of Digital Seismograph Networks. SEED Reference Manual, version 2.4. https://www.fdsn.org/publications/.

[10] Timescale, Inc. TimescaleDB documentation: hypertables and native compression. https://docs.timescale.com.

[11] pgvector contributors. pgvector: open-source vector similarity search for Postgres, version 0.8.0. https://github.com/pgvector/pgvector.

[12] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4, 824–836. doi:10.1109/TPAMI.2018.2889473.

[13] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proc. 2019 Int. Conf. on Management of Data (SIGMOD '19)*, 1981–1984. doi:10.1145/3299869.3320212.

[14] Apache Software Foundation. Apache Parquet format specification. https://parquet.apache.org/docs/.

[15] Michael Stonebraker, Paul Brown, Donghui Zhang, and Jacek Becla. 2013. SciDB: A Database Management System for Applications with Complex Analytics. *Computing in Science & Engineering* 15, 3, 54–62. doi:10.1109/MCSE.2013.19.

[16] Stavros Papadopoulos, Kushal Datta, Samuel Madden, and Timothy Mattson. 2016. The TileDB Array Data Storage Manager. *Proc. VLDB Endow.* 10, 4, 349–360. doi:10.14778/3025111.3025117.

[17] Clara E. Yoon, Ossian O'Reilly, Karianne J. Bergen, and Gregory C. Beroza. 2015. Earthquake detection through computationally efficient similarity search. *Science Advances* 1, 11, e1501057. doi:10.1126/sciadv.1501057.

[18] Thanawin Rakthanmanon, Bilson Campana, Abdullah Mueen, Gustavo Batista, Brandon Westover, Qiang Zhu, Jesin Zakaria, and Eamonn Keogh. 2012. Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping. In *Proc. 18th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD '12)*, 262–270. doi:10.1145/2339530.2339576.

[19] Chin-Chia Michael Yeh, Yan Zhu, Liudmila Ulanova, Nurjahan Begum, Yifei Ding, Hoang Anh Dau, Diego Furtado Silva, Abdullah Mueen, and Eamonn Keogh. 2016. Matrix Profile I: All Pairs Similarity Joins for Time Series: A Unifying View That Includes Motifs, Discords and Shapelets. In *IEEE 16th Int. Conf. on Data Mining (ICDM)*, 1317–1322. doi:10.1109/ICDM.2016.0179.

[20] Bob Kemp, Aeilko H. Zwinderman, Bert Tuk, Hilbert A. C. Kamphuisen, and Josefien J. L. Oberyé. 2000. Analysis of a sleep-dependent neuronal feedback loop: the slow-wave microcontinuity of the EEG. *IEEE Trans. Biomed. Eng.* 47, 9, 1185–1194. doi:10.1109/10.867928.

[21] Ary L. Goldberger, Luis A. N. Amaral, Leon Glass, Jeffrey M. Hausdorff, Plamen Ch. Ivanov, Roger G. Mark, Joseph E. Mietus, George B. Moody, Chung-Kang Peng, and H. Eugene Stanley. 2000. PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. *Circulation* 101, 23, e215–e220. doi:10.1161/01.CIR.101.23.e215.

[22] Patrick Wagner, Nils Strodthoff, Ralf-Dieter Bousseljot, Dieter Kreiseler, Fatima I. Lunze, Wojciech Samek, and Tobias Schaeffter. 2020. PTB-XL, a large publicly available electrocardiography dataset. *Scientific Data* 7, Article 154. doi:10.1038/s41597-020-0495-6.

[23] GFZ German Research Centre for Geosciences and Institut des Sciences de l'Univers–Centre National de la Recherche CNRS-INSU. 2006. IPOC Seismic Network. Integrated Plate boundary Observatory Chile. Network code CX. doi:10.14470/PK615318.

[24] Mohamed-Ali Tnani, Michael Feil, and Klaus Diepold. 2022. Smart Data Collection System for Brownfield CNC Milling Machines: A New Benchmark Dataset for Data-Driven Machine Monitoring. *Procedia CIRP* 107, 131–136. doi:10.1016/j.procir.2022.04.022.

[25] Christian Lessmeier, James Kuria Kimotho, Detmar Zimmer, and Walter Sextro. 2016. Condition Monitoring of Bearing Damage in Electromechanical Drive Systems by Using Motor Current Signals of Electric Motors: A Benchmark Data Set for Data-Driven Classification. In *Proc. European Conf. of the PHM Society (PHME 2016)*. doi:10.36001/phme.2016.v3i1.1577.

[26] J. Lee, H. Qiu, G. Yu, J. Lin, and Rexnord Technical Services. 2007. Bearing Data Set. IMS, University of Cincinnati. NASA Prognostics Data Repository, NASA Ames Research Center. https://data.nasa.gov/dataset/ims-bearings.

[27] MorletDB contributors. 2026. MorletDB source repository, protocols and results. https://github.com/merkhet-io/morletdb.

[28] MorletDB contributors. Plain PostgreSQL baselines P1 and P2. `evaluation/plain_postgres/README.md` in [27].

[29] MorletDB contributors. B1: lossless storage against modern baselines. `evaluation/storage_baselines/` in [27].

[30] MorletDB contributors. R1b confirmation (2022). `evaluation/materialized/compact/results/conf-2022/RESULTS.md` in [27].

[31] MorletDB contributors. R2: retrieval scaling, and R2-D diagnostic. `evaluation/retrieval_scaling/` in [27].

[32] MorletDB contributors. S1: sleep staging by retrieval. `evaluation/sleep/RESULTS.md` in [27].

[33] MorletDB contributors. Stage-1 compression benchmark. `evaluation/compression/results/ab70819/RESULTS.md` in [27].

[34] John Paparrizos and Luis Gravano. 2015. k-Shape: Efficient and Accurate Clustering of Time Series. In *Proc. 2015 ACM SIGMOD Int. Conf. on Management of Data (SIGMOD '15)*. doi:10.1145/2723372.2737793.

[35] Abdullah Mueen et al. The Fastest Similarity Search Algorithm for Time Series Subsequences under Euclidean Distance. https://www.cs.unm.edu/~mueen/FastestSimilaritySearch.html.

[36] Singh, Subramanya, Krishnaswamy, and Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. arXiv:2105.09613.

[37] Gong, Sun, Fang, Liu, Chen, and Gao. 2025. VStream: A Distributed Streaming Vector Search System. *Proc. VLDB Endow.* 18, 6, 1593–1606. doi:10.14778/3725688.3725692.

[38] MorletDB contributors. BL1: shifted-probe retrieval, confirmation and ingestion. `evaluation/bounded_lag/` in [27].

[39] MorletDB contributors. SS1: one stored envelope representation. `evaluation/search_storage/SS1_RESULTS.md` in [27].

[40] MorletDB contributors. SS3: filtered envelope search. `evaluation/search_storage/SS3_RESULTS.md` in [27].

[41] MorletDB contributors. Pilot stages 1–2: access control, recovery and load. `evaluation/pilot/RESULTS.md` in [27].

[42] MorletDB contributors. CW1: external compute workers. `evaluation/external_worker/CW1_RESULTS.md` in [27].
[43] MorletDB contributors. X1: reference coding across repeated operations. `evaluation/compression/reference/RESULTS.md` in [27].

A EVIDENCE LEDGER

Claim	Source	Restriction
6.12–11.53 bits/sample in DB	P2	one TimescaleDB configuration
24–35% larger (T int)	P2	users must supply counts
ALP ≈60 bits on calibrated	B1	DuckDB 2.0.0 implementation
1.5–1.6× below zstd	B1	2× claim not supported
1.3 / 6.7 bits of corrections	B1	two calibrated inputs
12.9% size, recall 0.9999	R1b	fidelity, not relevance
64.7% zero-lag overlap	P1	different similarity semantics
12 µs per window, linear	R2	measured range 4.2–235 k; warm, one client
zero-lag top 1000 + re-rank: recall 0.743	R2, R2-D	exact shortlist; R2 never used HNSW
HNSW top 1000 + re-rank: recall 0.744	R2-D	consumed data; one configuration
envelopes 12.9% vs 60.2% of raw	R1b, R2	stride, compressibility, layout
probes: recall 0.976, p95 131 vs 2,854 ms	BL1-C	held out; unfiltered; one machine
same-size zero-lag shortlist: 0.613	BL1-C	descriptive control
product function: 0.977, p95 50 ms	product check	same held-out queries
searchable at commit, 919k samples/s	BL1-S	development data; one day, 4 channels
filtered recall 0.515 → 1.0 at 1.8% eligible	SS3	routed to exact; 5 levels
router recall 0.977–1.0, no filter/RLS violations	SS3	latency bound missed by 0.8 ms (L5)
single representation saves 250 MB/235k windows	SS1	+23% exact or +27–32% probe p95
batched build: 1.31 → 0.31 ms DB CPU/window	CW1, #68	byte-identical; one machine
companion-table RLS: exact 946 → 995 ms	Stage 1	leak found and fixed; one tenant layout
recovery B1–B7 pass	Stage 2	50k windows; one machine
409.6k samples/s sustained, capacity 1.3–1.4×	C2, C3	939k windows; shared machine
WAL 13.8 → 1.91 GB per 15 min	C2, C2p	four settings changed together
κ 0.583 vs 0.541	S1	one EEG channel; no clinical claim